

CSE204 – Database Systems
Group Project – Final Report

RENTOGRAM

Airbnb-Based Accommodation Management System

Team Members	Course Information
Ömer Özenç Erkan Ceylan Emirhan Sak Muhammed Can Akbaş Yasin Taha İnal	Course: CSE204 – Database Systems Term: Spring 2026 Group: 17 Submission: April 2026

1. Executive Summary

RENTOGRAM is a full-stack, relational-database-driven accommodation management platform modeled after Airbnb. The system connects property owners (Hosts), travelers (Guests), and platform administrators through a single, normalized data architecture built entirely on MySQL.

This report documents the complete lifecycle of the database design: starting from a 75-attribute Unnormalized Form (UNF) raw dataset, progressing through 1NF, 2NF, and 3NF normalization stages, and arriving at a final schema of 11 optimized tables. It also describes all functional requirements, query implementations, role-based access control, the advanced payment and installment subsystem, multi-guest tracking, and an AI-powered review moderation layer.

The assignment received a score of 33/40 in its first submission. All instructor feedback has been addressed in this final version, including corrected ER diagram notation, improved review attribution, ALTER TABLE drop-column migrations, and three significant feature additions: multi-guest booking tracking (BookingGuests), registered card storage (UserCards), and installment-based payment processing (Installments).

2. Project Overview

2.1 System Purpose

RENTOGRAM provides a simple, reliable digital marketplace where property owners can list homes for short-term rental and guests can discover, book, and pay for accommodations securely. The platform is designed to replace fragmented paper-based or spreadsheet workflows with a scalable, integrity-enforced relational database system.

2.2 Core Workflow

Step	Actor	Action
1	Host	Adds a listing with property details, photos, amenities, and nightly pricing.
2	Guest	Searches available properties by city, date range, and amenity filters.
3	Guest	Selects dates, adds companion guests (BookingGuests), and confirms the reservation.
4	Guest	Pays via a registered card (UserCards) or chooses installment-based payment.

Step	Actor	Action
5	Guest	Leaves a star rating and written review after checkout. AI sentiment analysis is applied.

2.3 Target Users

User Type	Description
Individual Hosts	Property owners who list one or more homes and manage availability through a dashboard.
Property Management Companies	Businesses managing high volumes of properties requiring scalable multi-property analytics.
Guests	Travelers who search, book, and review properties with secure payment options.
Administrators	Platform operators with global reporting access and content moderation capabilities.

3. Instructor Feedback & Revisions

3.1 Assignment 1 Score Summary

Item	Max Points	Score Received	Notes
Item 1 (30+ Columns)	5	5	Full marks.
Item 2 (ER Diagram)	18	15	Participation notation was confusing; review attribution needed correction.
Item 3 (Normalization SQL)	12	8	ALTER TABLE drop-column statements were missing.
Item 4 (View Creation)	5	5	Full marks.
Total	40	33	

3.2 Changes Made in Response to Feedback

ER Diagram Corrections

The participation notation was revised so that the mandatory User participation in both Host and Guest subtypes is clearly expressed using correct (1,1) notation. The review relationship is now explicitly attributed to a Guest entity, not a generic User, resolving the question raised by the instructor.

ALTER TABLE Migrations Added

Section 5.4 of this report now includes explicit ALTER TABLE ... DROP COLUMN statements documenting how denormalized columns were removed from the UNF staging table as each normalized table was populated, completing the migration trail that was missing from the first submission.

New Features Implemented (Customer Request)

Feature	Implementation
Multi-guest booking tracking	New BookingGuests table stores companion traveler names, relationships, and ages per booking.
Customer payment card storage	New UserCards table stores masked card numbers per registered user (PCI-safe).
Installment-based payments	New Installments table splits a Payment into timed tranches with individual due dates and statuses.

Feature	Implementation
Guest list by property/month	hms_monthlyguesttracker VIEW delivers this report without ad-hoc joins.
Property list with host & booking count	hms_propertybookingsummary VIEW aggregates booking counts per property.
Change booking date	Parameterized UPDATE query with status guard (cannot modify completed/cancelled bookings).
Process a payment	Parameterized INSERT into Payments with automatic status set to completed.
Remove a booking	Parameterized DELETE using booking_id + guest_id; ON DELETE CASCADE handles child records.

4. Data Items & Attributes

The system manages 75 attributes distributed across 11 normalized tables. Below is the complete attribute inventory grouped by functional domain.

4.1 User & Authentication Domain

#	Attribute	Data Type	Table
1	user_id	INT (PK)	Users
2	full_name	VARCHAR(100)	Users
3	email	VARCHAR(150) UNIQUE	Users
4	phone	VARCHAR(20)	Users
5	password_hash	VARCHAR(255)	Users
6	role	ENUM(host, guest, admin)	Users
7	account_status	ENUM(active, suspended, deleted)	Users
8	created_at	DATETIME	Users
9	card_id	INT (PK)	UserCards
10	user_id (FK)	INT	UserCards
11	card_holder	VARCHAR(100)	UserCards
12	card_number_masked	VARCHAR(20)	UserCards
13	expiry_date	VARCHAR(7)	UserCards

4.2 Property Domain

#	Attribute	Data Type	Table
14	property_id	INT (PK)	Properties
15	host_id (FK)	INT	Properties
16	title	VARCHAR(200)	Properties
17	description	TEXT	Properties
18	property_type	VARCHAR(50)	Properties
19	room_type	VARCHAR(50)	Properties
20	city	VARCHAR(100)	Properties
21	district	VARCHAR(100)	Properties
22	address_line	TEXT	Properties
23	base_price	DECIMAL(10,2)	Properties

#	Attribute	Data Type	Table
24	cleaning_fee	DECIMAL(10,2)	Properties
25	max_guests	INT	Properties
26	bedroom_count	INT	Properties
27	bathroom_count	INT	Properties
28	status	ENUM(active, inactive)	Properties
29	created_at	DATETIME	Properties
30	photo_id	INT (PK)	PropertyPhotos
31	property_id (FK)	INT	PropertyPhotos
32	image_url	VARCHAR(500)	PropertyPhotos
33	is_primary	BOOLEAN	PropertyPhotos
34	sort_order	INT	PropertyPhotos
35	amenity_id	INT (PK)	Amenities
36	amenity_name	VARCHAR(100)	Amenities
37	amenity_icon	VARCHAR(100)	Amenities
38	property_id (FK)	INT	PropertyAmenities
39	amenity_id (FK)	INT	PropertyAmenities

4.3 Booking & Guest Domain

#	Attribute	Data Type	Table
40	booking_id	INT (PK)	Bookings
41	property_id (FK)	INT	Bookings
42	guest_id (FK)	INT	Bookings
43	check_in	DATE	Bookings
44	check_out	DATE	Bookings
45	guest_count	INT	Bookings
46	total_price	DECIMAL(10,2)	Bookings
47	status	ENUM(confirmed, cancelled, completed)	Bookings
48	created_at	DATETIME	Bookings
49	guest_info_id	INT (PK)	BookingGuests
50	booking_id (FK)	INT	BookingGuests
51	full_name	VARCHAR(100)	BookingGuests
52	relationship	VARCHAR(50)	BookingGuests
53	age	INT	BookingGuests

4.4 Payment & Financial Domain

#	Attribute	Data Type	Table
54	payment_id	INT (PK)	Payments
55	booking_id (FK)	INT	Payments
56	amount	DECIMAL(10,2)	Payments
57	payment_method	VARCHAR(50)	Payments
58	payment_status	ENUM(pending, completed, failed, refunded)	Payments
59	transaction_ref	VARCHAR(100)	Payments
60	payment_date	DATETIME	Payments
61	installment_id	INT (PK)	Installments
62	payment_id (FK)	INT	Installments
63	installment_number	INT	Installments
64	amount	DECIMAL(10,2)	Installments
65	due_date	DATE	Installments
66	status	ENUM(pending, paid, overdue)	Installments

4.5 Review Domain

#	Attribute	Data Type	Table
67	review_id	INT (PK)	Reviews
68	booking_id (FK)	INT	Reviews
69	guest_id (FK)	INT	Reviews
70	property_id (FK)	INT	Reviews
71	rating	TINYINT (1-5)	Reviews
72	comment	TEXT	Reviews
73	created_at	DATETIME	Reviews
74	review_date	DATE	Reviews
75	is_verified	BOOLEAN	Reviews

5. Normalization Process

5.1 UNF – Problems Identified

The initial dataset was a single flat table containing 75 attributes. The following anomalies were identified:

- Repeating Groups: Amenities, property photos, and additional guests repeated as separate columns within the same row (amenity_1, amenity_2, photo_1, photo_2, etc.).
- Update Anomaly: Changing a property base price or a user's phone number required updating every row that referenced that property or user.
- Insertion Anomaly: A new property or amenity could not be added to the system unless at least one booking existed.
- Deletion Anomaly: Deleting a booking record also destroyed the associated property and host information embedded in that row.

5.2 UNF → 1NF (First Normal Form)

Rules applied: all column values must be atomic; repeating groups must be removed; every row must be uniquely identifiable.

Actions taken:

- Extracted multi-valued amenity columns (amenity_1, amenity_2, amenity_3) into a separate Amenities table and a PropertyAmenities junction table.
- Extracted photo columns (photo_1, photo_2) into a separate PropertyPhotos table.
- Extracted compound guest data into a BookingGuests table for additional travelers per booking.
- Assigned a single-column primary key to every entity.

```
-- 1NF: Amenities - remove multi-valued amenity columns
INSERT INTO Amenities (name, icon)
SELECT DISTINCT unf_amenity_name, unf_amenity_icon
FROM UNF_Table WHERE unf_amenity_name IS NOT NULL;

-- 1NF: PropertyPhotos - separate repeating photo arrays
INSERT INTO PropertyPhotos (property_id, image_url, is_primary, sort_order)
SELECT DISTINCT property_id, unf_photo_url, unf_is_primary, unf_sort_order
FROM UNF_Table WHERE unf_photo_url IS NOT NULL;

-- 1NF: BookingGuests - isolate multiple guests per booking
INSERT INTO BookingGuests (booking_id, full_name, relationship, age)
SELECT DISTINCT booking_id, unf_guest_name, unf_guest_relation, unf_guest_age
FROM UNF_Table WHERE unf_guest_name IS NOT NULL;
```

5.3 1NF → 2NF (Second Normal Form)

Rules applied: no partial dependencies; all non-key attributes must depend on the entire primary key.

Partial dependencies found in the composite key (Booking_ID, Property_ID, User_ID):

- Property Level: {Property_ID} determines Title, City, Base_Price, Host_ID — these depend only on the property, not the booking.
- User Level: {User_ID} determines Full_Name, Email, Phone, Role — contact data depends only on the user.
- Booking Level: {Booking_ID} determines Check_In, Check_Out, Guest_Count, Total_Price, Status — independent of property or user.

```
-- 2NF: Users - eliminate partial dependency on user data
INSERT INTO Users (full_name, email, phone, password_hash, role, account_status)
SELECT DISTINCT unf_user_fullname, unf_user_email, unf_user_phone,
    unf_password_hash, unf_role, unf_account_status FROM UNF_Table;

-- 2NF: Properties - eliminate partial dependency on property data
INSERT INTO Properties
    (host_id, title, description, property_type, room_type, city,
    district, address_line, base_price, cleaning_fee,
    max_guests, bedroom_count, bathroom_count, status)
SELECT DISTINCT host_id, unf_prop_title, unf_prop_desc, unf_prop_type,
    unf_room_type, unf_city, unf_district, unf_address, unf_base_price,
    unf_cleaning_fee, unf_max_guests, unf_bedrooms, unf_bathrooms, unf_status
FROM UNF_Table;

-- 2NF: PropertyAmenities - resolve Many-to-Many (M:N) relationship
INSERT INTO PropertyAmenities (property_id, amenity_id)
SELECT DISTINCT property_id, amenity_id FROM UNF_Table
WHERE property_id IS NOT NULL AND amenity_id IS NOT NULL;
```

5.4 2NF → 3NF (Third Normal Form)

Rules applied: no transitive dependencies; non-key attributes must depend only on the primary key.

Transitive dependency chains removed:

- Payment Chain: Booking_ID → Payment_ID → Payment_Method, Payment_Status. Payment details moved to the Payments table.
- Installment Chain: Payment_ID → Installment_ID → Due_Date, Installment_Amount. Installment data moved to its own table.
- Review Chain: Booking_ID → Review_ID → Rating, Comment. Review data separated; guest_id added directly to Reviews to fix instructor feedback.

```
-- 3NF: Bookings - core transaction table
INSERT INTO Bookings
  (property_id, guest_id, check_in, check_out, guest_count, total_price, status)
SELECT DISTINCT property_id, guest_id, unf_check_in, unf_check_out,
  unf_guest_count, unf_total_price, unf_booking_status FROM UNF_Table;

-- 3NF: Payments - transitive dependency removed from Bookings
INSERT INTO Payments
  (booking_id, amount, payment_method, payment_status, transaction_ref)
SELECT DISTINCT booking_id, unf_payment_amount, unf_payment_method,
  unf_payment_status, unf_transaction_ref
FROM UNF_Table WHERE unf_payment_amount IS NOT NULL;

-- 3NF: Installments - transitive dependency removed from Payments
INSERT INTO Installments
  (payment_id, installment_number, amount, due_date, status)
SELECT DISTINCT payment_id, unf_installment_no, unf_installment_amount,
  unf_due_date, unf_installment_status
FROM UNF_Table WHERE unf_installment_no IS NOT NULL;

-- 3NF: Reviews - transitive dependency removed; guest_id linked directly
INSERT INTO Reviews
  (booking_id, guest_id, property_id, rating, comment)
SELECT DISTINCT booking_id, guest_id, property_id,
  unf_review_rating, unf_review_comment
FROM UNF_Table WHERE unf_review_rating IS NOT NULL;

-- 3NF: UserCards - card data separated from Users
INSERT INTO UserCards (user_id, card_holder, card_number_masked, expiry_date)
SELECT DISTINCT user_id, unf_card_holder, unf_card_masked, unf_card_expiry
FROM UNF_Table WHERE unf_card_masked IS NOT NULL;
```

5.5 ALTER TABLE: Column Removal (Missing from Assignment 1)

The following ALTER TABLE statements formally complete the migration by dropping denormalized columns from the UNF staging table once data has been migrated to normalized tables. This step was identified as missing in the instructor feedback.

```
-- Drop repeating amenity columns (migrated to Amenities + PropertyAmenities)
ALTER TABLE UNF_Table
  DROP COLUMN unf_amenity_name,
  DROP COLUMN unf_amenity_icon;

-- Drop repeating photo columns (migrated to PropertyPhotos)
ALTER TABLE UNF_Table
  DROP COLUMN unf_photo_url,
  DROP COLUMN unf_is_primary,
  DROP COLUMN unf_sort_order;

-- Drop guest info columns (migrated to BookingGuests)
```

```
ALTER TABLE UNF_Table
  DROP COLUMN unf_guest_name,
  DROP COLUMN unf_guest_relation,
  DROP COLUMN unf_guest_age;

-- Drop payment columns (migrated to Payments)
ALTER TABLE UNF_Table
  DROP COLUMN unf_payment_amount,
  DROP COLUMN unf_payment_method,
  DROP COLUMN unf_payment_status,
  DROP COLUMN unf_transaction_ref;

-- Drop installment columns (migrated to Installments)
ALTER TABLE UNF_Table
  DROP COLUMN unf_installment_no,
  DROP COLUMN unf_installment_amount,
  DROP COLUMN unf_due_date,
  DROP COLUMN unf_installment_status;

-- Drop review columns (migrated to Reviews)
ALTER TABLE UNF_Table
  DROP COLUMN unf_review_rating,
  DROP COLUMN unf_review_comment;

-- Drop card columns (migrated to UserCards)
ALTER TABLE UNF_Table
  DROP COLUMN unf_card_holder,
  DROP COLUMN unf_card_masked,
  DROP COLUMN unf_card_expiry;
```

6. Final Database Schema (11 Tables)

The 75-attribute UNF dataset was decomposed into 11 tables in Third Normal Form. The schema is implemented on MySQL with full foreign key constraints and ON DELETE CASCADE referential integrity.

Table	Primary Key / Foreign Keys	# Columns
Users	PK: user_id	8
UserCards	PK: card_id FK: user_id → Users	5
Properties	PK: property_id FK: host_id → Users	16
PropertyPhotos	PK: photo_id FK: property_id → Properties	5
Amenities	PK: amenity_id	3
PropertyAmenities	Composite PK: (property_id, amenity_id) both FKs	2
Bookings	PK: booking_id FK: property_id → Properties, guest_id → Users	9
BookingGuests	PK: guest_info_id FK: booking_id → Bookings	5
Payments	PK: payment_id FK: booking_id → Bookings	7
Installments	PK: installment_id FK: payment_id → Payments	6
Reviews	PK: review_id FK: booking_id, guest_id, property_id	9

6.1 Key Design Decisions

- Users is the single identity table for all actors (Host, Guest, Admin), differentiated by the role ENUM field. This eliminates the Hosts and Guests subtables used in Assignment 1, simplifying joins while retaining role-based access logic in the application layer.
- Reviews carries a direct guest_id FK (correcting the feedback issue) so that reviews are unambiguously attributed to the guest who made the booking.
- UserCards stores only masked card numbers to maintain PCI-DSS compliance. Full card processing is delegated to an external payment gateway.
- BookingGuests uses a 1:N relationship from Bookings, allowing any number of companion travelers to be recorded per reservation.
- Installments uses a 1:N relationship from Payments, splitting a single payment transaction into timed installment tranches.
- ON DELETE CASCADE is applied on all foreign keys so that deleting a User removes their Properties, which cascades to their Bookings, which cascades to Payments, Installments, and Reviews automatically.

7. SQL Query Implementations

7.1 View: Detailed Reservations

A master reporting view that joins Bookings, Users, Properties, and Payments into a single queryable dataset for administrators.

```
CREATE OR REPLACE VIEW HMS_DetailedReservations AS
SELECT
  b.booking_id,
  u.full_name           AS guest_name,
  p.title               AS property_title,
  p.city,
  b.check_in,
  b.check_out,
  b.total_price,
  b.status              AS booking_status,
  pay.payment_status,
  pay.amount            AS paid_amount
FROM Bookings b
JOIN Users u      ON b.guest_id = u.user_id
JOIN Properties p ON b.property_id = p.property_id
LEFT JOIN Payments pay ON b.booking_id = pay.booking_id;
```

7.2 Query 1 – List Guests by Property and Month

Returns all primary and additional guests who stayed at a given property during a specific month. Implemented as a reusable VIEW.

```
CREATE VIEW hms_monthlyguesttracker AS
SELECT
  p.property_id,
  p.title           AS property_title,
  u.full_name       AS primary_guest_name,
  bg.full_name      AS additional_guest_name,
  bg.relationship,
  b.check_in,
  MONTH(b.check_in) AS stay_month,
  YEAR(b.check_in)  AS stay_year
FROM bookings b
JOIN properties p      ON b.property_id = p.property_id
JOIN users u           ON b.guest_id = u.user_id
LEFT JOIN bookingguests bg ON b.booking_id = bg.booking_id;
```

7.3 Query 2 – List Properties with Hosts and Booking Count

Generates a property report showing host name and total booking statistics for each listing. Includes properties with zero bookings via LEFT JOIN.

```
CREATE VIEW hms_propertybookingsummary AS
SELECT
  p.property_id,
  p.title           AS property_title,
  u.full_name       AS host_name,
  COUNT(b.booking_id) AS total_bookings
FROM properties p
JOIN users u      ON p.host_id = u.user_id
LEFT JOIN bookings b ON p.property_id = b.property_id
GROUP BY p.property_id, p.title, u.full_name;
```

7.4 Query 3 – Change Booking Date (UPDATE)

Allows a guest to reschedule an active booking. The WHERE clause guards against modification of completed or cancelled reservations and verifies guest ownership.

```
UPDATE Bookings
SET
  check_in    = %s,
  check_out   = %s,
  total_price = %s
WHERE booking_id = %s
  AND guest_id   = %s
  AND status NOT IN ('completed', 'cancelled');
```

7.5 Query 4 – Process Payment for a Booking (INSERT)

Records a payment transaction for a booking, with optional installment splitting. The payment_status is set directly to 'completed' on successful transaction submission.

```
-- Record the main payment
INSERT INTO Payments
  (booking_id, amount, payment_method, payment_status, transaction_ref)
VALUES (%s, %s, %s, 'completed', %s);

-- Optionally split into installments (example: 3 installments)
INSERT INTO Installments
  (payment_id, installment_number, amount, due_date, status)
VALUES
  (LAST_INSERT_ID(), 1, %s, %s, 'pending'),
  (LAST_INSERT_ID(), 2, %s, %s, 'pending'),
  (LAST_INSERT_ID(), 3, %s, %s, 'pending');
```

7.6 Query 5 – Remove a Booking (DELETE)

Cancels and removes a booking record. ON DELETE CASCADE propagates the deletion to BookingGuests, Payments, Installments, and Reviews automatically, ensuring no orphaned records remain.

```
DELETE FROM Bookings
WHERE booking_id = %s
      AND guest_id = %s;

-- ON DELETE CASCADE automatically removes:
-- BookingGuests, Payments -> Installments, Reviews
```

8. Functional Dependencies

The following functional dependencies formed the basis for the normalization process:

Determinant	Dependent Attributes
user_id	full_name, email, phone, password_hash, role, account_status, created_at
card_id	user_id, card_holder, card_number_masked, expiry_date
property_id	host_id, title, description, property_type, room_type, city, district, address_line, base_price, cleaning_fee, max_guests, bedroom_count, bathroom_count, status
photo_id	property_id, image_url, is_primary, sort_order
amenity_id	amenity_name, amenity_icon
booking_id	property_id, guest_id, check_in, check_out, guest_count, total_price, status, created_at
guest_info_id	booking_id, full_name, relationship, age
payment_id	booking_id, amount, payment_method, payment_status, transaction_ref, payment_date
installment_id	payment_id, installment_number, amount, due_date, status
review_id	booking_id, guest_id, property_id, rating, comment, created_at, review_date, is_verified

Transitive chains that were resolved during 3NF normalization:

- Booking_ID → Payment_ID → Payment_Method, Payment_Status
- Payment_ID → Installment_ID → Due_Date, Installment_Amount
- Booking_ID → Review_ID → Rating, Comment (booking can exist without a review)

9. Role-Based Access Control

The system supports three distinct user roles, each with a customized dashboard and restricted data access. Role logic is enforced at the application layer using the role field in the Users table.

Role	Capabilities
Admin	View global reports (hms_citystatistics, hms_successfulhosts). Manage all users, properties, and reservations. Moderate AI-flagged reviews. Access all analytics dashboards.
Host	Add, edit, and deactivate their own property listings. View bookings for their properties. Access property-level booking summaries and occupancy analytics.
Guest	Search and filter available properties. Create, update, and cancel bookings. Register payment cards. Submit post-stay reviews. View their booking history.

10. Advanced Features

10.1 Multi-Guest Tracking (BookingGuests)

Families and travel groups often require more than one guest to be registered per booking. The BookingGuests table implements a 1:N relationship from Bookings, allowing each reservation to track an unlimited number of additional travelers with their name, relationship to the primary guest (e.g., spouse, child), and age.

Design Rationale

Storing companion guest data in a separate table rather than as repeating columns in Bookings is the correct 1NF approach. The LEFT JOIN in the monthlyguesttracker VIEW ensures primary guests without companions are still returned in guest listings.

10.2 Secure Card Storage (UserCards)

Guests can register one or more payment cards to their profile. Only masked card numbers (e.g., **** * 4242) are stored in the database. Full card processing is handled by a PCI-DSS-compliant external payment gateway. The UserCards table has a 1:N relationship with Users, so one guest can store multiple cards.

10.3 Installment Payments (Installments)

The Installments table extends the Payments system to support timed payment plans. A single Payment record can be split into N installments, each with an `installment_number`, `due_date`, per-installment amount, and status (pending, paid, overdue). This enables the platform to offer buy-now-pay-later accommodation booking without complicating the core Payments or Bookings tables.

10.4 Review Verification & Date Tracking

Each review entry records the exact date it was submitted via the `review_date` (DATE) field, enabling chronological sorting, time-based filtering, and audit trails. The `is_verified` (BOOLEAN) flag indicates whether the review was submitted by a guest who completed an actual, confirmed stay at the property. Only bookings with a status of 'completed' are eligible to leave a verified review. Unverified reviews (e.g., submitted before checkout is confirmed) are flagged accordingly, protecting property owners from premature or fraudulent feedback.

11. Database Views Summary

Six SQL views encapsulate the most common reporting queries, making them reusable without repeated JOIN logic:

View Name	Purpose
hms_detailedreservations	Master booking report: joins Bookings, Users, Properties, and Payments into one flat dataset.
hms_monthlyguesttracker	Lists all primary and additional guests per property per month (Query 1).
hms_propertybookingsummary	Property report with host name and total booking count (Query 2).
hms_propertyanalytics	Aggregate occupancy and revenue metrics per property.
hms_citystatistics	Booking volume and average ratings grouped by city for admin dashboards.
hms_successfulhosts	Ranks hosts by total bookings and average review score.

12. End-to-End Booking Workflow

The complete guest-facing booking process follows five sequential steps, each backed by a database operation:

Step	Action	Database Operation
1. Search	Guest filters properties by city, dates, and amenities.	SELECT with WHERE and JOIN against Properties, PropertyAmenities, and Bookings (date overlap check).
2. Select	Guest views listing details, pricing calendar, and photos.	SELECT from Properties, PropertyPhotos, Amenities, Reviews.
3. Book	Guest reserves dates, adds companion travelers.	INSERT into Bookings + BookingGuests.
4. Pay	Guest selects a registered card or chooses installments.	INSERT into Payments + optional INSERT into Installments.
5. Confirm	Booking status updated; guest receives confirmation.	UPDATE Bookings SET status = 'confirmed'.

13. Conclusion

RENTOGRAM demonstrates the practical power of disciplined relational database design. By transforming a flat 75-attribute dataset through 1NF, 2NF, and 3NF normalization stages, the project arrives at an 11-table schema that is free from data redundancy, resistant to update/insertion/deletion anomalies, and fully enforced by foreign key constraints.

The instructor feedback from Assignment 1 has been fully addressed: the ER diagram participation notation is corrected, review records are now directly attributed to Guest entities, and the previously missing ALTER TABLE drop-column migrations are now documented. The three customer-requested features (multi-guest tracking, card storage, and installment payments) have been implemented as first-class normalized entities rather than application-layer hacks.

The addition of six reporting views, AI-assisted review moderation, and a role-based access control model makes RENTOGRAM a production-ready blueprint for a modern short-term rental management platform.

Achievement	Detail
Normalization	75-attribute UNF dataset normalized to 11 3NF tables.
Data Integrity	ON DELETE CASCADE across all FK relationships; no orphaned records.
New Features	BookingGuests, UserCards, Installments tables fully implemented.
Query Coverage	5 parameterized DML queries + 6 reporting VIEWS delivered.
Feedback Resolution	ER notation corrected, review attribution fixed, ALTER TABLE added.
Review Integrity	review_date and is_verified fields ensure chronological tracking and verified-stay attribution.